

Reverse LL in group of given size K

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list:
```

```
struct ListNode
```

```
{
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode()
```

```
{
```

```
    val = 0;
```

```
    next = NULL;
```

```
}
```

```
    ListNode(int data1)
```

```
{
```

```
    val = data1;
```

```
    next = NULL;
```

```
}
```

```
    ListNode(int data1, ListNode *next1)
```

```
{
```

```
    val = data1;
```

```
    next = next1;
```

```
}
```

```
};
```

```
class Solution {
```

```
public:
```

```
    // Function to reverse a linked list
```

```

// Using the 3-pointer approach
ListNode* reverseLinkedList(ListNode *head)
{
    /* Initialize 'temp' at
    * head of linked list */
    ListNode* temp = head;

    /* Initialize pointer 'prev'
    * to NULL, representing
    * the previous node */
    ListNode* prev = NULL;

    // Continue till 'temp'
    // reaches the end (NULL)
    while(temp != NULL){
        /* Store the next node in 'front'
        * to preserve the reference */
        ListNode* front = temp->next;

        /* Reverse the direction of the
        * current node's 'next' pointer
        * to point to 'prev' */
        temp->next = prev;

        /* Move 'prev' to the current
        * node for the next iteration */
        prev = temp;

        /* Move 'temp' to the 'front' node
        * advancing the traversal */
        temp = front;
    }
}

```

```

    }

    // Return the new head
    // of the reversed linked list
    return prev;
}

// Function to get the Kth node from a
// given position in the linked list
ListNode* getKthNode(ListNode* temp, int k){
    // Decrement K
    // as we already start
    // from the 1st node
    k -= 1;

    // Decrement K until it reaches the desired position
    while(temp != NULL && k > 0){
        // Decrement k as temp progresses
        k--;

        // Move to the next node
        temp = temp -> next;
    }

    // Return the Kth node
    return temp;
}

// Function to reverse nodes in groups of K
ListNode* reverseKGroup(ListNode* head, int k){
    /* Initialize a temporary

```

```

* node to traverse the list */
ListNode* temp = head;

/* Initialize a pointer to track
* the last node of the previous group */
ListNode* prevLast = NULL;

// Traverse through the linked list
while(temp != NULL){
    // Get the Kth node of the current group
    ListNode* kThNode = getKthNode(temp, k);

    /* If the Kth node is NULL
    * (not a complete group) */
    if(kThNode == NULL){
        /* If there was a previous group,
        * link the last node to the current node */
        if(prevLast){
            prevLast -> next = temp;
        }

        // Exit the loop
        break;
    }

    /* Store the next node
    * after the Kth node */
    ListNode* nextNode = kThNode -> next;

    /* Disconnect the Kth node
    * to prepare for reversal */

```

```

kThNode -> next = NULL;

// Reverse the nodes from temp to the Kth node
reverseLinkedList(temp);

/* Adjust the head if the reversal
 * starts from the head */
if(temp == head){
    head = kThNode;
}else{
    /* Link the last node of the previous
     * group to the reversed group */
    prevLast -> next = kThNode;
}

/* Update the pointer to the
 * last node of the previous group */
prevLast = temp;

// Move to the next group
temp = nextNode;
}

// Return the head of the modified linked list
return head;
}
};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;

```

```

while (temp != nullptr) {
    cout << temp->val << " ";
    temp = temp->next;
}
cout << endl;
}

int main() {
    // Create a linked list with values 5, 4, 3, 7, 9 and 2
    ListNode* head = new ListNode(5);
    head->next = new ListNode(4);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(7);
    head->next->next->next->next = new ListNode(9);
    head->next->next->next->next->next = new ListNode(2);

    // Print the original linked list
    cout << "Original Linked List: ";
    printLinkedList(head);

    // Reverse the linked list in groups of K
    Solution solution;
    head = solution.reverseKGroup(head, 4);

    // Print the reversed linked list
    cout << "Reversed Linked List: ";
    printLinkedList(head);

    return 0;
}

```

Rotate a LL

```
#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode() {
        val = 0;
        next = NULL;
    }
    ListNode(int data1) {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1) {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to rotate the list by k steps
    ListNode* rotateRight(ListNode* head, int k) {
        // Base case: if list is empty or has only one node
        if (head == NULL || head->next == NULL) return head;

        // Perform rotation k times
```

```

for (int i = 0; i < k; i++) {
    ListNode* temp = head;

    // Find the second last node
    while (temp->next->next != NULL)
        temp = temp->next;

    // Get the last node
    ListNode* end = temp->next;

    // Break the link between
    // second last and last node
    temp->next = NULL;

    // Make the last node
    // as new head
    end->next = head;

    head = end;
}

return head;
};

// Utility function to insert node at the end of the list
void insertNode(ListNode* &head, int val) {
    ListNode* newNode = new ListNode(val);

    if (head == NULL) {
        head = newNode;
        return;
    }

    ListNode* temp = head;
    while (temp->next != NULL) temp = temp->next;

    temp->next = newNode;
}

```



```

// Utility function to print list
void printList(ListNode* head) {
    while (head != NULL) {
        cout << head->val;
        if (head->next != NULL) cout << "->";
        head = head->next;
    }
    cout << endl;
}

int main() {
    ListNode* head = NULL;

    // Inserting nodes
    insertNode(head, 1);
    insertNode(head, 2);
    insertNode(head, 3);
    insertNode(head, 4);
    insertNode(head, 5);

    cout << "Original list: ";
    printList(head);

    int k = 2;
    Solution solution;

    // Calling function for rotating right by k times
    ListNode* newHead = solution.rotateRight(head, k);

    cout << "After " << k << " iterations: ";

    // List after rotating nodes
    printList(newHead);
}

```

```
    return 0;
}
```

```
#include <iostream>

using namespace std;
```

```
// Definition of singly linked list
```

```
class ListNode {
public:
    int val;
    ListNode* next;
    ListNode() {
        val = 0;
        next = nullptr;
    }
    ListNode(int data1) {
        val = data1;
        next = nullptr;
    }
    ListNode(int data1, ListNode* next1) {
        val = data1;
        next = next1;
    }
};
```

```
class Solution {
public:
    // Function to rotate the list by k steps
    ListNode* rotateRight(ListNode* head, int k) {
        if (head == nullptr || head->next == nullptr || k == 0)
            return head;
```

```

// Calculating length
ListNode* temp = head;
int length = 1;
while (temp->next != nullptr) {
    ++length;
    temp = temp->next;
}

// Link last node to first node
temp->next = head;

// When k is more than length of list
k = k % length;

// To get end of the list
int end = length - k;
while (end-- > 0)
    temp = temp->next;

// Breaking last node link and pointing to NULL
head = temp->next;
temp->next = nullptr;

return head;
}
};

// Utility function to insert node at the end of the list
void insertNode(ListNode*& head, int val) {
    ListNode* newNode = new ListNode(val);
    if (head == nullptr) {
        head = newNode;
    }
}

```

```

        return;
    }
    ListNode* temp = head;
    while (temp->next != nullptr) temp = temp->next;
    temp->next = newNode;
}

```

// Utility function to print list

```

void printList(ListNode* head) {
    while (head != nullptr) {
        cout << head->val;
        if (head->next != nullptr) cout << "->";
        head = head->next;
    }
    cout << endl;
}

```

```

int main() {
    ListNode* head = new ListNode(1);
    // Inserting nodes
    insertNode(head, 2);
    insertNode(head, 3);
    insertNode(head, 4);
    insertNode(head, 5);

```

```

    cout << "Original list: ";
    printList(head);

```

```

    int k = 2;

```

```

    Solution solution;

```

```

    // Calling function for rotating right by k times

```

```

ListNode* newHead = solution.rotateRight(head, k);

cout << "After " << k << " iterations: ";

// List after rotating nodes
printList(newHead);

return 0;
}

```

Merge two Sorted Lists

```

#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int data1) {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1) {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to merge two sorted linked lists
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {

```

```

vector<int> arr;

ListNode* temp1 = list1;
ListNode* temp2 = list2;


// Add elements from list1 to the vector
while(temp1 != NULL){
    arr.push_back(temp1->val);
    // Move to the next node in list1
    temp1 = temp1->next;
}


// Add elements from list2 to the vector
while(temp2 != NULL){
    arr.push_back(temp2->val);
    // Move to the next node in list2
    temp2 = temp2->next;
}


// Sorting the vector in ascending order
sort(arr.begin(), arr.end());


// Sorted vector to linked list
ListNode* dummyNode = new ListNode(-1);
ListNode* temp = dummyNode;
for(int i = 0; i < arr.size(); i++){
    temp->next = new ListNode(arr[i]);
    temp = temp->next;
}


// Return the head of

```

```

        // merged sorted linked list
        return dummyNode->next;
    }
};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        // Print the data of the current node
        cout << temp->val << " ";

        // Move to the next node
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Example Linked Lists
    ListNode* list1 = new ListNode(1);
    list1->next = new ListNode(3);
    list1->next->next = new ListNode(5);

    ListNode* list2 = new ListNode(2);
    list2->next = new ListNode(4);
    list2->next->next = new ListNode(6);

    cout << "First sorted linked list: ";
    printLinkedList(list1);

    cout << "Second sorted linked list: ";

```

```
printLinkedList(list2);
```

```
Solution solution;
```

```
ListNode* mergedList = solution.mergeTwoLists(list1, list2);
```

```
cout << "Merged sorted linked list: ";
```

```
printLinkedList(mergedList);
```

```
return 0;
```

```
}
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode(int data1) {
```

```
        val = data1;
```

```
        next = NULL;
```

```
    }
```

```
    ListNode(int data1, ListNode *next1) {
```

```
        val = data1;
```

```
        next = next1;
```

```
    }
```

```
};
```

```
class Solution {
```

```
public:
```



```

// Function to merge two sorted linked lists

ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    // Create a dummy node to serve as
    // the head of the merged list
    ListNode* dummyNode = new ListNode(-1);
    ListNode* temp = dummyNode;

    // Traverse both lists simultaneously
    while (list1 != nullptr && list2 != nullptr) {
        /*Compare elements of both lists
        and link the smaller node
        to the merged list*/
        if (list1->val <= list2->val) {
            temp->next = list1;
            list1 = list1->next;
        } else {
            temp->next = list2;
            list2 = list2->next;
        }
        // Move the temporary pointer
        // to the next node
        temp = temp->next;
    }

    /*If any list still
    has remaining elements,
    append them to the merged list*/
    if (list1 != nullptr) {
        temp->next = list1;
    } else {
        temp->next = list2;
    }
}

```

```

    }

    // Return merged list
    return dummyNode->next;
}

};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        // Print the data of the current node
        cout << temp->val << " ";

        // Move to the next node
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Example Linked Lists
    ListNode* list1 = new ListNode(1);
    list1->next = new ListNode(3);
    list1->next->next = new ListNode(5);

    ListNode* list2 = new ListNode(2);
    list2->next = new ListNode(4);
    list2->next->next = new ListNode(6);

    cout << "First sorted linked list: ";
    printLinkedList(list1);

```

```
cout << "Second sorted linked list: ";  
printLinkedList(list2);
```

```
Solution solution;
```

```
ListNode* mergedList = solution.mergeTwoLists(list1, list2);
```

```
cout << "Merged sorted linked list: ";  
printLinkedList(mergedList);
```

```
return 0;
```

```
}
```

Flattening of LL

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of special linked list:
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode *child;
```

```
    ListNode() {
```

```
        val = 0;
```

```
        next = NULL;
```

```
        child = NULL;
```

```
    }
```

```
    ListNode(int data1) {
```

```
        val = data1;
```

```
        next = NULL;
```

```

        child = NULL;
    }
    ListNode(int data1, ListNode *next1, ListNode* next2) {
        val = data1;
        next = next1;
        child = next1;
    }
};

```

```

class Solution {
private:
    // Function to convert a vector to a linked list
    ListNode* convertArrToLinkedList(vector<int>& arr) {
        /* Create a dummy node to serve as
        the head of the linked list */
        ListNode* dummyNode = new ListNode(-1);
        ListNode* temp = dummyNode;

        /* Iterate through the vector and
        create nodes with vector elements */
        for (int i=0; i < arr.size(); i++) {
            // Create a new node with the vector element
            temp->child = new ListNode(arr[i]);

            // Update the temporary pointer
            temp = temp->child;
        }

        /* Return the linked list starting
        from the next of the dummy node */
        return dummyNode->child;
    }
};

```

```
}
```

```
public:
```

```
// Function to flatten a linked list with child pointers
```

```
ListNode* flattenLinkedList(ListNode* head) {
```

```
    vector<int> arr;
```

```
    // Traverse through the linked list
```

```
    while (head != nullptr) {
```

```
        /* Traverse through the child
```

```
        nodes of each head node */
```

```
        ListNode* t2 = head;
```

```
        while (t2 != nullptr) {
```

```
            // Store each node's data in the array
```

```
            arr.push_back(t2->val);
```

```
            // Move to the next child node
```

```
            t2 = t2->child;
```

```
        }
```

```
        // Move to the next head node
```

```
        head = head->next;
```

```
    }
```

```
    // Sort the array containing node values
```

```
    sort(arr.begin(), arr.end());
```

```
    // Convert the sorted array back to a linked list
```

```
    return convertArrToLinkedList(arr);
```

```
}
```

```
};
```

```
// Function to print the linked list
```

```
void printLinkedList(ListNode* head) {  
    while (head != nullptr) {  
        cout << head->val << " ";  
        head = head->child;  
    }  
    cout << endl;  
}
```

```
// Function to print the linked list in a grid-like structure
```

```
void printOriginalLinkedList(ListNode* head, int depth) {  
    while (head != nullptr) {  
        cout << head->val;  
  
        /* If child exists, recursively  
        print it with indentation */  
        if (head->child) {  
            cout << " -> ";  
            printOriginalLinkedList(head->child, depth + 1);  
        }  
  
        // Add vertical bars for each level in the grid  
        if (head->next) {  
            cout << endl;  
            for (int i = 0; i < depth; ++i) {  
                cout << "| ";  
            }  
        }  
        head = head->next;  
    }  
}
```

```
}
```

```
int main() {
```

```
    // Create a linked list with child pointers
```

```
    ListNode* head = new ListNode(5);
```

```
    head->child = new ListNode(14);
```

```
    head->next = new ListNode(10);
```

```
    head->next->child = new ListNode(4);
```

```
    head->next->next = new ListNode(12);
```

```
    head->next->next->child = new ListNode(20);
```

```
    head->next->next->child->child = new ListNode(13);
```

```
    head->next->next->next = new ListNode(7);
```

```
    head->next->next->next->child = new ListNode(17);
```

```
    // Print the original linked list structure
```

```
    cout << "Original linked list:" << endl;
```

```
    printOriginalLinkedList(head, 0);
```

```
    // Creating an instance of Solution class
```

```
    Solution sol;
```

```
    // Function call to flatten the linked list
```

```
    ListNode* flattened = sol.flattenLinkedList(head);
```

```
    // Printing the flattened linked list
```

```
    cout << "\nFlattened linked list: ";
```

```
    printLinkedList(flattened);
```

```
    return 0;
}
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of special linked list
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode *child;
```

```
    ListNode() {
```

```
        val = 0;
```

```
        next = NULL;
```

```
        child = NULL;
```

```
    }
```

```
    ListNode(int data1) {
```

```
        val = data1;
```

```
        next = NULL;
```

```
        child = NULL;
```

```
    }
```

```
    ListNode(int data1, ListNode *next1, ListNode* next2) {
```

```
        val = data1;
```

```
        next = next1;
```

```
        child = next1;
```

```
    }
```

```
};
```

```
class Solution {
```

```
private:
```

```
    /* Merge the two linked lists in a particular
```



```

order based on the data value */
ListNode* merge(ListNode* list1, ListNode* list2){
    /* Create a dummy node as a
    placeholder for the result */
    ListNode* dummyNode = new ListNode(-1);
    ListNode* res = dummyNode;

    // Merge the lists based on data values
    while(list1 != NULL && list2 != NULL){
        if(list1->val < list2->val){
            res->child = list1;
            res = list1;
            list1 = list1->child;
        }
        else{
            res->child = list2;
            res = list2;
            list2 = list2->child;
        }
        res->next = NULL;
    }

    // Connect the remaining elements if any
    if(list1){
        res->child = list1;
    } else {
        res->child = list2;
    }

    // Break the last node's link to prevent cycles
    if(dummyNode->child){

```

```

        dummyNode->child->next = NULL;
    }

    return dummyNode->child;
}

public:
    // Function to flatten a linked list with child pointers
    ListNode* flattenLinkedList(ListNode* head) {
        // If head is null or there is no next node
        if(head == NULL || head->next == NULL){
            return head; // Return head
        }

        // Recursively flatten the rest of the linked list
        ListNode* mergedHead = flattenLinkedList(head->next);

        // Merge the lists
        head = merge(head, mergedHead);
        return head;
    }
};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    while (head != nullptr) {
        cout << head->val << " ";
        head = head->child;
    }
    cout << endl;
}

```

```

// Function to print the linked list in a grid-like structure
void printOriginalLinkedList(ListNode* head, int depth) {
    while (head != nullptr) {
        cout << head->val;

        /* If child exists, recursively
        print it with indentation */
        if (head->child) {
            cout << " -> ";
            printOriginalLinkedList(head->child, depth + 1);
        }

        // Add vertical bars for each level in the grid
        if (head->next) {
            cout << endl;
            for (int i = 0; i < depth; ++i) {
                cout << " | ";
            }
        }
        head = head->next;
    }
}

```

```

int main() {
    // Create a linked list with child pointers
    ListNode* head = new ListNode(5);
    head->child = new ListNode(14);

    head->next = new ListNode(10);
    head->next->child = new ListNode(4);
}

```

```

head->next->next = new ListNode(12);
head->next->next->child = new ListNode(20);
head->next->next->child->child = new ListNode(13);

head->next->next->next = new ListNode(7);
head->next->next->next->child = new ListNode(17);

// Print the original linked list structure
cout << "Original linked list:" << endl;
printOriginalLinkedList(head, 0);

// Creating an instance of Solution class
Solution sol;

// Function call to flatten the linked list
ListNode* flattened = sol.flattenLinkedList(head);

// Printing the flattened linked list
cout << "\nFlattened linked list: ";
printLinkedList(flattened);

return 0;
}

```

Sort LL

```

#include <bits/stdc++.h>

using namespace std;

```

```
// Definition of singly linked list
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode() : val(0), next(NULL) {}
```

```
    ListNode(int data1) : val(data1), next(NULL) {}
```

```
    ListNode(int data1, ListNode *next1) : val(data1), next(next1) {}
```

```
};
```

```
class Solution {
```

```
public:
```

```
    // Function to sort Linked List
```

```
    ListNode* sortList(ListNode* head) {
```

```
        // Vector to store node values
```

```
        vector<int> arr;
```

```
        /*Temporary pointer to
```

```
        traverse the linked list*/
```

```
        ListNode* temp = head;
```

```
        /* Traverse the linked list and
```

```
        store node values in the vector*/
```

```
        while(temp != NULL) {
```

```
            arr.push_back(temp->val);
```

```
            temp = temp->next;
```

```
        }
```

```
        // Sort array containing node values
```

```
        sort(arr.begin(), arr.end());
```

```

// Reassign sorted values to linked list nodes

temp = head;
for(int i = 0; i < arr.size(); i++) {
    // Update the node's data
    temp->val = arr[i];
    // Move to the next node
    temp = temp->next;
}

// Return the head
return head;
}
};

```

```

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        // Print the data of the current node
        cout << temp->val << " ";
        // Move to the next node
        temp = temp->next;
    }
    cout << endl;
}

```

```

int main() {
    // Linked List: 3 2 5 4 1
    ListNode* head = new ListNode(3);
    head->next = new ListNode(2);

```

```

head->next->next = new ListNode(5);
head->next->next->next = new ListNode(4);
head->next->next->next->next = new ListNode(1);

cout << "Original Linked List: ";
printLinkedList(head);

Solution solution;
// Sort the linked list
head = solution.sortList(head);

cout << "Sorted Linked List: ";
printLinkedList(head);

return 0;
}

#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(NULL) {}
    ListNode(int data1) : val(data1), next(NULL) {}
    ListNode(int data1, ListNode *next1) : val(data1), next(next1) {}
};

class Solution {

```

public:

// Function to merge two sorted linked lists

ListNode* mergeTwoSortedLinkedLists(ListNode* list1, ListNode* list2) {

// Create dummy node to serve as head of merged list

ListNode* dummyNode = new ListNode(-1);

ListNode* temp = dummyNode;

// Traverse both lists simultaneously

while (list1 != nullptr && list2 != nullptr) {

/*Compare elements of both lists

and link the smaller node

to the merged list*/

if (list1->val <= list2->val) {

temp->next = list1;

list1 = list1->next;

} else {

temp->next = list2;

list2 = list2->next;

}

// Move the temporary pointer to the next node

temp = temp->next;

}

/*If any list still has

remaining elements append

them to the merged list*/

if (list1 != nullptr) {

temp->next = list1;

} else {

temp->next = list2;

}


```
// Return the merged list starting
// from the next of the dummy node
return dummyNode->next;
}
```

```
// Function to find the middle of a linked list
ListNode* findMiddle(ListNode* head) {
    // If the list is empty or has only one node,
    // the middle is the head itself
    if (head == nullptr || head->next == nullptr) {
        return head;
    }
```

```
// Initializing slow and fast pointers
ListNode* slow = head;
ListNode* fast = head->next;
```

```
/* Move the fast pointer twice as fast as the slow pointer
When the fast pointer reaches the end, the slow pointer
will be at the middle*/
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

```
return slow;
}
```

```
// Function to perform merge sort on a linked list
ListNode* sortList(ListNode* head) {
    /*Base case: if the list is empty or has only one node,
```

```

it is already sorted, so return the head*/
if (head == nullptr || head->next == nullptr) {
    return head;
}

// Find middle of list using findMiddle function
ListNode* middle = findMiddle(head);

// Divide the list into two halves
ListNode* right = middle->next;
middle->next = nullptr;
ListNode* left = head;

// Recursively sort left and right halves
left = sortList(left);
right = sortList(right);

// Merge the sorted halves using the
// mergeTwoSortedLinkedLists function
return mergeTwoSortedLinkedLists(left, right);
}
};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        // Print the data of the current node
        cout << temp->val << " ";
        // Move to the next node
        temp = temp->next;
    }
}

```

```

    }

    cout << endl;
}

int main() {
    // Linked List: 3 2 5 4 1
    ListNode* head = new ListNode(3);
    head->next = new ListNode(2);
    head->next->next = new ListNode(5);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new ListNode(1);

    cout << "Original Linked List: ";
    printLinkedList(head);

    Solution solution;
    // Sort the linked list
    head = solution.sortList(head);

    cout << "Sorted Linked List: ";
    printLinkedList(head);

    return 0;
}

```

Clone a LL with random and next pointer

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

```

struct ListNode {
    int val;
    ListNode *next;
    ListNode *random;
    ListNode() {
        val = 0;
        next = NULL;
        random = NULL;
    }
    ListNode(int data1) {
        val = data1;
        next = NULL;
        random = NULL;
    }
    ListNode(int data1, ListNode *next1, ListNode* r) {
        val = data1;
        next = next1;
        random = r;
    }
};

```

```

class Solution {
public:
    // Function to clone linked list with random pointers
    ListNode* copyRandomList(ListNode* head) {
        // If the head is null, return null
        if (!head) return NULL;

        /*Create an unordered_map to map
        original nodes to their corresponding copied nodes*/
        unordered_map<ListNode*, ListNode*> mpp;
    }
};

```

```
ListNode* temp = head;

// Create copies of each node
while (temp != NULL) {
    // Create new node with same value as original
    ListNode* newNode = new ListNode(temp->val);
    // Map to original node
    mpp[temp] = newNode;
    // Move to next node
    temp = temp->next;
}
```

```
// Reset temp
temp = head;
```

```
/*Connect the next and
random pointers of the
copied nodes using the map*/
while (temp != NULL) {
    // Get copied node from the map
    ListNode* copyNode = mpp[temp];
    /*Set next pointer of copied node
to the copied node of the next
original node*/
    copyNode->next = mpp[temp->next];
    /*Set the random pointer of the
copied node to the copied node of
the random original node*/
    copyNode->random = mpp[temp->random];
    temp = temp->next;
}
```

```

        // Return the head
        return mpp[head];
    }
};

// Function to print the linked list
void printClonedLinkedList(ListNode *head) {
    while (head != nullptr) {
        // Print the data of the current node
        cout << "Data: " << head->val;

        // Print the data of the random pointer, if it exists
        if (head->random != nullptr) {
            cout << ", Random: " << head->random->val;
        } else {
            cout << ", Random: nullptr";
        }

        cout << endl;

        // Move to the next node
        head = head->next;
    }
}

```

```

int main() {
    // Example linked list: 7 -> 14 -> 21 -> 28
    ListNode* head = new ListNode(7);
    head->next = new ListNode(14);
    head->next->next = new ListNode(21);
    head->next->next->next = new ListNode(28);

    // Assigning random pointers

```

```

head->random = head->next->next; // 7 -> 21
head->next->random = head; // 14 -> 7
head->next->next->random = head->next->next->next; // 21 -> 28
head->next->next->next->random = head->next; // 28 -> 14

// Print the original linked list
cout << "Original Linked List with Random Pointers:" << endl;
printClonedLinkedList(head);

// Clone the linked list
Solution solution;
ListNode* clonedList = solution.copyRandomList(head);

// Print the cloned linked list
cout << "\nCloned Linked List with Random Pointers:" << endl;
printClonedLinkedList(clonedList);

return 0;
}

#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode *random;
    ListNode() {
        val = 0;
        next = NULL;
        random = NULL;
    }
};

```

```

    }

    ListNode(int data1) {
        val = data1;
        next = NULL;
        random = NULL;
    }

    ListNode(int data1, ListNode *next1, ListNode* r) {
        val = data1;
        next = next1;
        random = r;
    }
};

class Solution {
public:
    // Insert a copy of each node in between the original nodes
    void insertCopyInBetween(ListNode* head) {
        ListNode* temp = head;
        while (temp != NULL) {
            ListNode* nextElement = temp->next;
            // Create a new node with the same data
            ListNode* copy = new ListNode(temp->val);

            copy->next = nextElement;

            temp->next = copy;

            temp = nextElement;
        }
    }
};

```



```
// Function to connect random pointers of the copied nodes
```

```
void connectRandomPointers(ListNode* head) {  
    ListNode* temp = head;  
    while (temp != NULL) {  
        // Access the copied node  
        ListNode* copyNode = temp->next;  
  
        /*If the original node has a random pointer  
        point the copied node's random to the  
        corresponding copied random node  
        set the copied node's random to null  
        if the original random is null*/  
        if (temp->random) {  
  
            copyNode->random = temp->random->next;  
        } else {  
  
            copyNode->random = NULL;  
        }  
  
        // Move to next original node  
        temp = temp->next->next;  
    }  
}
```

```
// Function to retrieve the deep copy of the linked list
```

```
ListNode* getDeepCopyList(ListNode* head) {  
    ListNode* temp = head;  
    // Create a dummy node  
    ListNode* dummyNode = new ListNode(-1);  
    // Initialize a result pointer
```

```

ListNode* res = dummyNode;

while (temp != NULL) {
    /*Creating a new List by
    pointing to copied nodes*/
    res->next = temp->next;
    res = res->next;

    /*Disconnect and revert back
    to the initial state of the
    original linked list*/
    temp->next = temp->next->next;
    temp = temp->next;
}

/*Return the deep copy
of the list starting
from the dummy node*/
return dummyNode->next;
}

// Function to clone the linked list
ListNode* copyRandomList(ListNode* head) {
    // If the original list is empty, return null
    if (!head) return nullptr;

    // Insert nodes in between
    insertCopyInBetween(head);

    // Connect random pointers
    connectRandomPointers(head);

    // Retrieve deep copy of linked list

```

```

        return getDeepCopyList(head);
    }
};

// Function to print the cloned linked list
void printClonedLinkedList(ListNode* head) {
    while (head != nullptr) {
        cout << "Data: " << head->val;
        if (head->random != nullptr) {
            cout << ", Random: " << head->random->val;
        } else {
            cout << ", Random: nullptr";
        }
        cout << endl;
        // Move to the next node
        head = head->next;
    }
}

int main() {
    // Example linked list: 7 -> 14 -> 21 -> 28
    ListNode* head = new ListNode(7);
    head->next = new ListNode(14);
    head->next->next = new ListNode(21);
    head->next->next->next = new ListNode(28);

    // Assigning random pointers
    head->random = head->next->next; // 7 -> 21
    head->next->random = head; // 14 -> 7
    head->next->next->random = head->next->next->next; // 21 -> 28
    head->next->next->next->random = head->next; // 28 -> 14
}

```

```
// Print the original linked list
cout << "Original Linked List with Random Pointers:" << endl;
printClonedLinkedList(head);

// Clone the linked list
Solution solution;
ListNode* clonedList = solution.copyRandomList(head);

// Print the cloned linked list
cout << "\nCloned Linked List with Random Pointers:" << endl;
printClonedLinkedList(clonedList);

return 0;
}
```